Week 6 · Lab 1

Review — More Loops and First Functions: `do-while`, `for`, nested loops, functions

`do-while` · `for` · nested loops · writing a function · `return` versus `cout` · `void` · `<cmath>`

Week: 6 — first session (review, led by the TA)
Duration: 2 hours 15 minutes
Format: the TA explains Lecture 5 again, then practice
Tools: computer with CLion; pen for Sections A–F
Prerequisite: Lecture 5; Week 5 labs

Name:

Group:

Date:

Learning outcomes

By the end of this lab you should be able to:

- **LO1.** Write `do-while` loops and say when the body runs at least once.
- **LO2.** Write `for` loops, trace them with a table, and choose between `for`, `while` and `do-while`.
- **LO3.** Trace nested loops and count their passes (outer × inner).
- **LO4.** Write a function with a return type, parameters and `return`, and call it from `main`.
- **LO5.** Explain the difference between `return` and `cout`, and when a function is `void`.
- **LO6.** Use `sqrt`, `pow`, `floor`, `ceil`, `round` from `<cmath>`.
- **LO7.** Recognise the classic bugs: `for (...);`, off-by-one bounds, a missing `;` after `do-while`, printing instead of returning.

This is the review session of Week 6: the TA goes through Lecture 5 again (*do-while, for, nested loops and the first functions*) and solves problems on it together with the class. Week 6 · Lab 2 is the graded practice on the same material.

How this lab is organised

This is a **review** session, **led by the TA**. In **Part I** the TA explains each idea of Lecture 5 at the board with the worked example; after each idea you solve the short **Quick check** at once, and the TA discusses the answer. In **Part II** the TA solves the *first item* of every section A–F on the board with the class and explains the reasoning; you finish the rest yourself. Sections A–F are on *paper*; Sections G–J are done in **CLion**.

Block	Content		Time
–	Introduction and learning outcomes		5 min
PART I — THE TA EXPLAINS, WITH WORKED EXAMPLES			40 min
§1	`do-while`		6 min
§2	`for` and its trace table		7 min
§3	Four `for` patterns and three `for` traps		5 min
§4	Which loop?		2 min
§5	Nested loops		6 min
§6	Writing a function		7 min
§7	`return` versus `cout`; `void`; `<cmath>`		5 min
§8	Common mistakes		2 min
–	*Short break*		5 min
PART II — PRACTICE (first item of A–F with the TA)			80 min
A	Trace `do-while` and `for`	*paper*	10 min
B	Nested loops	*paper*	8 min
C	Functions: predict the output	*paper*	8 min
D	Find the mistake	*paper*	6 min
E	Rewrite it	*paper*	6 min
F	True/false and multiple choice	*paper*	5 min
G	Average with input checking (`do-while`, `for`)	*CLion*	9 min
H	Table and triangle (nested `for`)	*CLion*	10 min
I	Temperature table (functions)	*CLion*	9 min
J	Number tools menu (functions + menu)	*CLion*	9 min
K	Bonus challenges *(not timed)*		—
–	Final checklist and wrap-up		5 min
Total			**2 h 15 min**

How your work is checked. Nothing is uploaded. Write the answers to Sections A–F on this sheet; do Sections G–J in CLion, one `.cpp` file per task, and run every test input. **When you have finished, raise your hand** and show the TA your sheet and each program running. The TA marks the score table on the last page.

PART I — The TA explains: theory and worked examples

1 do-while: run first, ask afterwards

```
int age;
do {
    cout << "Age (0-120): ";
    cin >> age;
} while (age < 0 || age > 120);
```

The test is at the **bottom**, so the body always runs **at least once** — and the closing `while (...)` needs a `;`. Read the condition as "the reason to ask again": *while the answer is still wrong*.

while or do-while?

`while` — "maybe zero times" (the test comes first). `do-while` — "at least once, then decide": input validation, a menu that reappears. With `n = 0`, `int i = 1; while (i <= n) {...}` prints nothing, while the same loop as `do-while` prints once.

Quick check 1. (a) `int i = 5; do { cout << i << " "; i -= 2; } while (i > 0); cout << "| " << i;`
(b) `int n = 0, s = 0; do { s += n; n++; } while (n < 0); cout << s << " " << n;`

2 for and its trace table

`for (initialise; test; update)` collects the three parts of every loop on one line. The initialisation runs once, the test before every pass, the update after every pass. A variable declared in the header lives only inside the loop.

Worked example 2.1 — one row per pass, plus the row where the test fails

```
int p = 1;
for (int i = 1; i <= 4; i++) {
    p *= i;
    cout << i << ":" << p << " ";
}
```

pass	i	i <= 4	p
1	1	true	1
2	2	true	2
3	3	true	6
4	4	true	24
–	5	false	24

Output: `1:1 2:2 3:6 4:24`. The last row — the test that fails — is the one people forget.

Quick check 2. (a) `int p = 1; for (int i = 2; i <= 8; i += 3) p *= i; cout << p;` (b) How many passes: `for (int i = 20; i > 0; i /= 2)`?

3 Four for patterns and three traps

```
for (int i = 5; i >= 1; i--)      // 5 4 3 2 1
for (int i = 0; i <= 10; i += 2)  // 0 2 4 ... 10
for (char c = 'a'; c <= 'e'; c++) // a b c d e
int f = 1;
for (int i = 1; i <= 5; i++) f *= i;   // 120
```

Traps. (1) `for (...);` — the `;` is the whole body. (2) Off by one: `i < 5` is 4 passes, `i <= 5` is 5. (3) Changing the counter inside the body (`i++` again) skips values. And never count with a `double`: `x != 1.0` with `x += 0.1` never stops.

4 Which loop?

Loop	Test is	Use it when
`while`	before the body	the number of passes depends on the data; zero passes is legal
`do-while`	after the body	the body must run at least once — validation, menus
`for`	before the body	you know how many passes: counting, stepping, tables

5 Nested loops

The inner loop runs **completely** for every pass of the outer one: the passes multiply. If the inner limit depends on the outer counter, the rows change length — every "shape" exercise is this idea.

```
for (int r = 1; r <= 4; r++) {
    for (int c = 1; c <= 4; c++) {
        cout << setw(4) << r * c;
    }
    cout << endl;     // ends the row
}
```

A 4×4 table: 16 inner passes. `setw(4)` (from `<iomanip>`) gives every number 4 columns. Two loops of 1000 each are 10^6 passes — nested loops are the first place where a program feels slow.

Quick check 3. (a) `for (int r = 1; r <= 3; r++) { for (int c = r; c <= 3; c++) cout << c; cout << " "; }`
(b) How many times does `steps++` run? `for (int i = 1; i <= 4; i++) for (int j = 1; j <= i; j++) steps++;`

6 Writing a function

```
int square(int x) {        // definition
    return x * x;
}

int main() {
    int n;
    cin >> n;
    cout << square(n) << endl;     // call
    cout << square(3) + square(4); // 25
    return 0;
}
```

Return type `int` — the type of the value handed back. **Name** `square`. **Parameter** `int x` — receives the caller's value. **Body** — runs only when called. `return` hands the value back *and ends the function*.

On a call, `main` pauses, the body runs with `x` = the argument, and `main` continues with the returned value. For now the definition stands **above** `main`: a name must be known before it is used.

Quick check 4. With `int add3(int x) { return x + 3; }`, what does `cout << add3(4) * add3(1);` print?

7 return versus cout; void; <cmath>

```
int twice(int x) {        // returns
    return 2 * x;
}
void show(int x) {        // only prints
    cout << 2 * x << " ";
}
int y = twice(5) + twice(3);   // 16
// int z = show(5);            // ERROR
```

A function that **returns** a value can be used inside an expression. A `void` function returns nothing — call it as a statement of its own. Rule: *calculate in the function, print in* `main`.

`<cmath>`: `sqrt(81)` 9, `pow(2, 10)` 1024, `floor(7.8)` 7, `ceil(7.2)` 8, `round(2.5)` 3 — all `double`.

Quick check 5. (a) With `show` and `twice` above: `show(5); cout << twice(5) + 1;` (b) `cout << sqrt(49) + pow(2, 3) << " " << floor(-2.5) << " " << round(-2.5);`

8 Common mistakes — and how to catch them

Mistake	Symptom and fix
`for (...);`	The `;` is the body; the block below runs once (or does not compile). Delete it.
`do { ... } while (cond)` without `;`	Compile error — this loop ends with `;`.
`<` versus `<=`	One pass too few or too many. Count the passes for a small case.
Inner counter set outside the outer loop	Only the first row is printed. Declare it in the outer loop.
Printing inside a function that should return	The value cannot be used in an expression; printed twice when `main` prints too.
Function written below `main`	"not declared in this scope" — move it above (prototypes come in Lecture 6).
`total / n` with two `int`s	Integer division. Write `total / (n * 1.0)`.

PART II — Practice

A Trace do-while and for

[10 min] (12 points)

Write the output **and** how many times the loop body runs. ($1\frac{1}{2}$ + $1\frac{1}{2}$ each) *The TA solves a) on the board with a trace table.*

a)

```
int x = 1;
do {
    x *= 2;
} while (x < 20);
cout << x;
```

Output: _____

Passes: _____

b)

```
int x = 50;
do {
    x -= 7;
} while (x > 50);
cout << x;
```

Output: _____

Passes: _____

c)

```
int s = 0;
for (int i = 1; i <= 10; i += 3) {
    s += i;
}
cout << s;
```

Output: _____

Passes: _____

d)

```
int s = 0;
for (int i = 1; i <= 6; i++) {
    if (i % 3 == 0) continue;
    s += i;
}
cout << s;
```

Output: _____

Passes: _____

B Nested loops

[8 min] (10 points)

Write exactly what each fragment prints. (2 each) *The TA solves a) on the board.*

a)

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 2; j++) {
        cout << i * j << " ";
    }
}
```

b)

```
for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= i; j++) {
        cout << "*";
    }
    cout << endl;
}
```

c)

```
int c = 0;
for (int i = 1; i <= 5; i++)
    for (int j = 1; j <= 5; j++)
        if (i < j) c++;
cout << c;
```

d)

```
for (char r = 'a'; r <= 'c'; r++) {
    for (char c = 'a'; c <= r; c++) {
        cout << c;
    }
    cout << " ";
}
```

e)

```
int c = 0;
for (int i = 1; i <= 4; i++) {
    int j = i;
    while (j > 0) {
        c++;
        j -= 2;
    }
}
cout << c;
```

C Functions: predict the output

[8 min] (10 points)

Each fragment shows the function(s) and the lines inside `main`. (2 each) *The TA solves a) on the board.*

a)

```
int sq(int x) {
    return x * x;
}
// in main:
cout << sq(3) + sq(sq(2));
```

b)

```
int f(int x) {
    cout << "f" << x << " ";
    return x + 1;
}
// in main:
int y = f(1);
y = f(y);
cout << y;
```

c)

```
double avg(int a, int b) {
    return (a + b) / 2;
}
// in main:
cout << avg(3, 4);
```

d)

```cpp
int first(int n) {
    for (int i = 1; i <= n; i++) {
        if (i * i > n) return i;
    }
    return -1;
}
// in main:
cout << first(20) << " " << first(0);
```

e)

```cpp
void line(int n) {
    for (int i = 0; i < n; i++) cout << "-";
    cout << "|";
}
// in main:
line(3);
line(0);
line(1);
```

D Find the mistake

[6 min] (8 points)

Each fragment has exactly one fault. Name it, say syntax or logic, and fix it. (2 each) *The TA solves a) on the board.*

a) Intended: print 1 to 5.

```cpp
for (int i = 1; i <= 5; i++);
    cout << i << " ";
```

b) Intended: a 3×3 multiplication table, one row per line.

```cpp
int c = 1;
for (int r = 1; r <= 3; r++) {
    while (c <= 3) {
        cout << r * c << " ";
        c++;
    }
    cout << endl;
}
```

c) Intended: return the larger of two numbers.

```cpp
int larger(int a, int b) {
    if (a > b) cout << a;
    else cout << b;
}
```

d) Intended: ask for a positive number until one is entered.

```cpp
do {
    cin >> n;
} while (n > 0);
```

E Rewrite it

[6 min] (8 points)

Rewrite each fragment in the form asked for. (2 each) *The TA solves a) on the board.*

a) As a **`for`** loop: `int i = 10; while (i >= 0) { cout << i << " "; i -= 2; }`

b) As a **`while`** loop: `for (int k = 1; k <= n; k++) { total += k * k; }`

c) As a **function** `double circleArea(double r)`, used three times: `double a1 = 3.14159 * 2 * 2; double a2 = 3.14159 * 5 * 5; double a3 = 3.14159 * 7 * 7;`

d) So that the function **returns** the result and `main` prints it: `void cube(int x) { cout << x * x * x; }`

F True/false and multiple choice

[5 min] (8 points)

Mark **T** or **F**; for every **F** write one line why. Then circle one answer. (1 each) *The TA solves statement 1 with the class.*

1. The body of a `do-while` loop always runs at least once.

2. In `for (init; test; update)`, the update runs before the first test.

3. A function's parameter exists only while that call runs.

4. A `void` function can be used inside `int y = f(3) + 1;`

5. How many passes: `for (int i = 0; i < 10; i += 2)`?
(a) 4 (b) 5 (c) 6 (d) 10

6. `int i = 10; do { i++; } while (i < 5);` — `i` is
(a) 10 (b) 11 (c) 5 (d) 6

7. `cout << pow(2, 10);` prints
(a) 20 (b) 1024 (c) 1024.0 (d) 100

8. Best loop for a menu that must appear at least once?
(a) `for` (b) `while` (c) `do-while` (d) none

G Average with input checking

[9 min] (10 points)

Task G — one `.cpp` file

Ask how many marks there are, repeating the question until the answer is at least 1 (`do-while`). Then read the marks with a `for` loop; ask for each mark again until it is from 0 to 100 (a `do-while` inside the `for`). Print the average with two decimals, the highest and the lowest mark.

Test input: `0 3 80 105 95 -5 70` (0, 105 and −5 are rejected)

```
Average: 81.67
Highest: 95
Lowest:  70
```

H Table and triangle

[10 min] (12 points)

Task H — one `.cpp` file

Read n (1–9). Print the $n \times n$ multiplication table with every number in 4 columns (`setw(4)`), then an empty line, then a right-aligned triangle of stars of height n.

Test input: `4`

```
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16

   *
  **
 ***
****
```

Hint: row r of the triangle has n − r spaces, then r stars — two inner loops, one after the other.

I Temperature table

[9 min] (10 points)

Task I — one `.cpp` file

Write `double toFahrenheit(double c)` ($F = C \cdot 9/5 + 32$) and `void printRow(int c)`, which prints one table row using `toFahrenheit`. In `main`, read `from`, `to` and `step` and print the table with one decimal (`fixed`, `setprecision(1)`), using a `for` loop that calls `printRow`.

Test input: `0 100 25`

```
   C      F
   0   32.0
  25   77.0
  50  122.0
  75  167.0
 100  212.0
```

With `-10 10 7`: the rows are −10, −3, 4 with F = _____.

J Number tools menu

[9 min] (12 points)

Task J — one .cpp file

Write `int gcd(int a, int b)` (Euclid: repeat `r = a % b; a = b; b = r;` until `b` is 0), `int lcm(int a, int b)` (using `gcd`) and `long long power(int base, int exp)` (a `for` loop). In `main`, show a menu in a `do-while` loop: `1 a b` prints the gcd, `2 a b` the lcm, `3 a b` the power a^b, `0` prints `Bye`; anything else prints `Unknown choice`.

Test the whole sequence: `1 12 18 2 4 6 3 2 10 7 0` → `gcd = 6`, `lcm = 12`, `power = 1024`, `Unknown choice`, `Bye`.

K Bonus challenges (up to 10 points, not timed)

1. (4) **Diamond.** Read n and print a diamond of stars with $2n - 1$ rows (the widest row has $2n - 1$ stars). Test `3`.

2. (3) **Armstrong numbers.** A k-digit number is an Armstrong number if the sum of the k-th powers of its digits equals the number ($153 = 1^3 + 5^3 + 3^3$). Write `countDigits`, `power` and `isArmstrong`; print all Armstrong numbers from 1 to n. Test `1000`.

3. (3) **Approximating π.** $\pi = 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots\right)$. Write `double leibnizPi(int terms)` and print the approximation with 6 decimals. Test `10`, `1000`, `100000`.

Final checklist [5 min]

- ☐ Your **name** and **group** are on the front page.
- ☐ Sections A–F are answered **on paper**, before you used a computer.
- ☐ No `for (...)` is followed by `;`, and every `do-while` ends with `;`.
- ☐ You counted the passes of each loop for a small input (`<` versus `<=`).
- ☐ Every function that computes a value **returns** it; printing happens in `main`.
- ☐ Every function is written above the place where it is called.
- ☐ Each program was run with **every** test input in its task.
- ☐ You **raised your hand** and showed the TA this sheet and each program running.

Section	Available	Earned
A Trace `do-while` and `for`	12	
B Nested loops	10	
C Functions: predict the output	10	
D Find the mistake	8	
E Rewrite it	8	
F True/false and multiple choice	8	
G Average with input checking	10	
H Table and triangle	12	
I Temperature table	10	
J Number tools menu	12	
Total	**100**	
K Bonus challenges	+10	

Checked by TA: _____

What comes next

- **Week 6 · Lab 2** is the graded practice: LeetCode problems solved as LeetCode functions, checked at your desk.
- Lecture 6 continues with functions — prototypes, references and overloading — practised in the Week 7 labs.

The habit worth keeping from today: **compute in the function, print in `main`.**

End of Week 6 · Lab 1